ALPHATESTER

MUSIC GENERATOR

Installation Guide for Mac and Windows

Create your own songs locally – no limits, no subscription, no cloud.

1. **Get the project**
   via Git or as a folder
2. **Run Install**
   double-click, the rest runs by itself
3. **Make music**
   Music Generator ON

Based on ACE-Step 1.5 (MIT license) via acestep.cpp · runs entirely locally, no account needed

BEFORE YOU START

At a Glance

Music Generator creates songs directly on your computer. You describe style and mood, the app composes studio-quality music from it (WAV, 48 kHz). Everything runs locally: no cloud, no download limits, your material stays on your machine.

What you need

	Requirement	Why
Mac	Apple Silicon (M1 or newer)	Sound generation uses the graphics chip. Intel Macs run on the CPU only and are very slow. Pages 3 and 4.
Windows	Windows 10/11, NVIDIA graphics card	RTX 20xx/GTX 16xx or newer runs on CUDA (not yet tested in practice), older cards on Vulkan (slower). Page 5.
Memory	at least 8 GB RAM, 16 GB recommended	Determines which language model fits (see page 4). On Windows, graphics memory counts too.
Disk	7 to 11 GB free, about 5 GB more with XL	Models, model server and room for your songs.
Internet	for the installation	The models (4 to 8 GB) are downloaded once. After that everything works offline.
Time	about 15 to 30 minutes	Depends mostly on your download speed.

How the installation works

1

Get the project

Download the project folder via Git or receive it as a folder.

2

Run Install

Double-click “Mac-Install” or “Windows-Install”. The installer checks your computer and sets everything up.

3

Make music

“Music Generator ON” opens the app in your browser.

Good to know: The installer hardly asks anything. You can run it again at any time: finished steps are skipped and interrupted downloads resume.

Installation videos: Windows: youtu.be/se2g3tRFzrg · Mac: youtu.be/FxNSXj_wMBQ

GET THE PROJECT, RUN THE INSTALLER

Steps 1 and 2 on the Mac

1 Get the project

Open the **Terminal** app (Spotlight: Cmd+Space, type “Terminal”) and paste this command. It downloads the project into the folder **MusicGenerator** in your home folder:

```
git clone https://github.com/rabbitfiremediacreation/music-generator.git \
    ~/MusicGenerator
```

If the Apple developer tools are missing, macOS now asks automatically whether to install them: confirm with **Install**, wait, then run the command again.

Without Git: If you received the project folder as a ZIP or via AirDrop, unpack it to a permanent place, for example your home folder. Don't leave it in Downloads, that folder tends to get cleaned up.

2 Run the installer

Open the folder in Finder and **double-click “Mac-Install”**. A Terminal window opens in which the installer works. Alternatively in the Terminal:

```
open ~/MusicGenerator/Mac-Install.command
```



The installer shows what it found on your Mac and suggests the matching language model. **Enter** accepts the recommendation, 1, 2 or 3 lets you choose. Then Enter once more to start, the rest runs by itself.

> **macOS warning:** If macOS only offers “Move to Trash” or “Cancel”: choose **Cancel**, then open **System Settings › Privacy & Security**, scroll to the bottom and click **Open Anyway** next to “Mac-Install” (password or Touch ID). Then double-click Install again. The Terminal route is on page 4.

AUTOMATICALLY, STEP BY STEP (MAC)

What the installer does

1. **Check the Mac**
   Detect chip, memory, macOS version and free space.
2. **Choose a model**
   Suggest the matching language model size, check disk space.
3. **Tools**
   Apple developer tools (for git) and uv (Python), without password and without Homebrew.
4. **Get the model server**
   Ready-made program files in the tested version, checksum-verified. Before macOS 26 it is compiled locally.
5. **Download models**
   Only the files your Mac needs, with progress bar, resumable.
6. **Test the graphics chip**
   Trial run: does the language model compute correctly on the graphics chip? Otherwise it uses the CPU.
7. **Set up icons**
   Green start and red stop icon for “Music Generator ON/OFF”.
8. **Go**
   Start the servers and open the app in the browser.

Which model?

0.6B
from 8 GB RAM
total download 4.4 GB

1.7B MOST COMMON
from 16 GB RAM
total download 5.7 GB

4B
from 24 GB RAM
total download 8.1 GB

Bigger means: song structure, tempo and lyrics are matched better. The sound model (SFT, high quality) is the same for all.

How long?

Get model server	under 1 min.
Download models	depends on speed
Graphics chip test	1 to 2 min.
One song (1:30)	about 2 to 4 min.

If macOS blocks the file

This happens with files from the internet, especially a ZIP. Instead of going through System Settings (page 3), one Terminal command fixes the whole folder in one go. Afterwards Install and Music Generator ON/OFF open without a prompt:

```
xattr -dr com.apple.quarantine ~/MusicGenerator
```

With **git clone** (step 1) the warning never appears in the first place.

STEPS 1 AND 2 ON THE PC

Installation on Windows

1 Get the project

Open **PowerShell** (Start menu, type “PowerShell”) and paste this command. It downloads the project into the folder **MusicGenerator** in your user folder:

```
git clone https://github.com/rabbitfiremediacreation/music-generator.git `
    $HOME\MusicGenerator
```

If Windows doesn't know the **git** command, run **winget install Git.Git** once first and reopen PowerShell. A ZIP works too: unpack it and put the folder in a permanent place.

2 Run the installer

In the **MusicGenerator** folder, **double-click “Windows-Install”** (the file Windows-Install.bat). A PowerShell window opens; the questions are the same as on the Mac: choose the language model, confirm with Enter, wait. Alternatively without questions:

```
.\Windows-Install.ps1 -Yes
```

The installer needs no admin rights and no compiler: it downloads ready-made program files of the model server (about 130 MB) and the models, verifies their checksums, tests the graphics card and places **Music Generator ON** and **OFF** as shortcuts in the folder and on the desktop.

> **Windows:** If “Open File – Security Warning” appears: click **Run**. This only happens with a downloaded ZIP, not after git clone.

What's different on Windows

Topic	Windows
Graphics card	RTX 20xx/GTX 16xx and newer run on CUDA according to file analysis (not yet tested in practice). Older cards (GTX 10xx) automatically use Vulkan: a 30-second song takes about 1 minute on a GTX 1060, 90 seconds about 2.5 minutes.
Language model	Graphics memory limits the choice as well: below 6 GB 0.6B, 6 to 7 GB 1.7B, from 8 GB 4B. The installer suggests a fitting one.
Start and stop	Shortcuts on the desktop and in the folder. OFF stops the servers but leaves the browser tab open.
Log files	In the data folder: server.log, ace-server.log and an .err.log with error messages for each.

From here on it's the same as on the Mac, from page 6. Help with Windows problems: page 9.

USING THE APP

Start, stop, get going

Music Generator ON

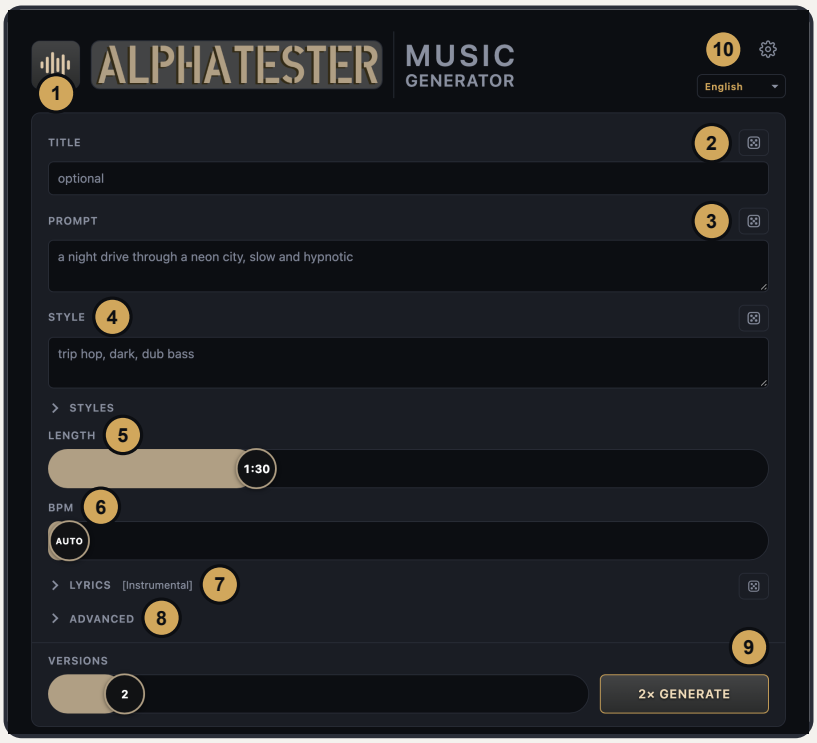
starts everything and opens the app in the browser

Music Generator OFF

stops everything and closes the browser tabs

Both are in the project folder, on Windows also on the desktop. Mac tip: drag them to the Dock. The app runs at **http://localhost:8765**, the windows close by themselves after starting. Shortcuts: **Alt+G** generate, **Alt+N** prompt, **Alt+B** library. Works with VoiceOver and NVDA, details in the README.

The screen



1. **Logo** · Hidden upload field: drop a song on it or click. The app analyzes it and fills in prompt, style, tempo and length.
2. **Title** · Only for finding it again. The dice suggests a name.
3. **Prompt** · Your idea: mood, scene, sound. The dice rolls an idea.
4. **Style** · Genre and instruments. Dice: suggestion. Under “Styles”, cards add terms.
5. **Length** · 10 seconds to 10 minutes, default 1:30. Above 5 minutes it takes long and needs a lot of memory.
6. **BPM** · Tempo. Far left = Auto, the model decides.
7. **Lyrics** · Collapsed = instrumental. Dice: “Automatic”, the model writes its own lyrics when generating. Expanded: language and your own lyrics.
8. **Advanced** · Iterations (IT, default 80: more = cleaner, takes longer), variance (VAR, default 1.2: higher = freer, lower = closer to style and prompt), key, time signature, seed.
9. **Generate** · Pick the number of versions (1 to 10) at the bottom and go. Several are named A, B, C … The bar stays visible while scrolling.
10. **Gear** · Settings: song folder, sound model (standard or optional XL), export (WAV or MP3), layout (stacked or side by side), “Check for updates”, stop servers (press twice). Below: interface language (twelve languages, from German to Chinese).

Record an idea: Press and hold the header's microphone button (5 to 30 s) and hum, whistle or sing. The app takes over description, tempo and key; you choose style and length. Not the melody itself. Nothing is stored.

HOW TO GET GOOD SONGS

Writing prompts and lyrics

What the model sees

The app sends **Style** and **Prompt** to the model as one line, style first, then prompt, word for word and without rewriting. Whatever is not there, the model does not know. It understands **English** best, and short comma-separated terms. Say what should be in the song, not what should be missing: “no drums” hardly works, “only piano and strings” does.

What belongs in style and prompt

Building block	Examples
Genre, one or two	deep house · trip hop · acoustic folk · cinematic orchestral
Mood and scene	melancholic, rainy night · euphoric festival · late-night drive through a neon city
Instruments and sound	warm Rhodes piano, 808 bass · distorted guitars · analog synths, tape hiss
Tempo and energy	slow and hypnotic · driving · builds to a big finale (plus the BPM slider)
Voice	female vocals · male rapper · soft choir. Leave out for instrumentals: every word about voice or singing produces voices.
Era and production	80s synthpop · lo-fi, dusty samples · live recording, small club

Instrumental or vocals

Instrumental: leave Lyrics collapsed, nothing else needed. Also keep voice words out of the prompt (vocal chops, ad-libs, rapper, choir), otherwise someone sings or talks. **Vocals:** expand Lyrics. The dice lets the model write its own words, which works well in English. In other languages it likes to invent nonsense: then write the text yourself or have a chat program write it and paste it in. Set the **language** in the selector under “Lyrics”, not in the prompt.

Structuring lyrics

The buttons above the field insert sections: **[verse]**, **[chorus]**, **[bridge]**, **[intro]**, **[outro]**. The model uses them as a blueprint: the chorus may repeat, the verses tell the story. Short lines sing better than long sentences. As a rule of thumb, about eight to ten lines fit into one minute; two verses and two choruses are enough for 1:30, three verses and a bridge for three minutes. Choose the **Length** to match the text, otherwise the model rushes or stretches.

Example

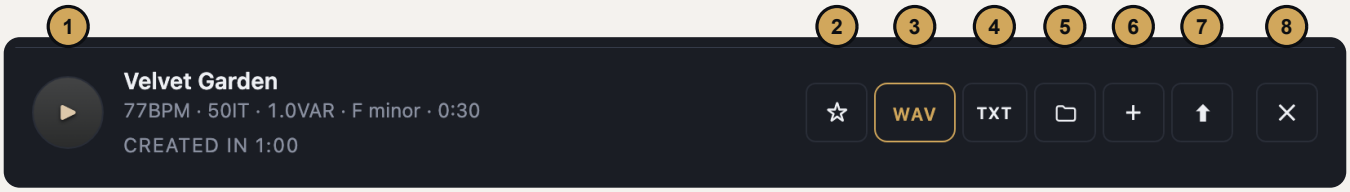
```
Style:   trip hop, dark, dub bass, female vocals
Prompt:  late-night drive through a neon city, slow and hypnotic
Lyrics:  [verse]  Headlights paint the rain in gold …
         [chorus] Keep on driving, keep on driving …
```

Refining: If a song is almost right: use the **arrow** to load its settings into the form, change only the text and generate again with the same **Seed** (Advanced), then the character stays similar. If the song feels scattered, lower **Variance**. Several **Versions** at once save time when comparing.

ONCE THE FIRST SONG IS DONE

Library, Folders, Updates

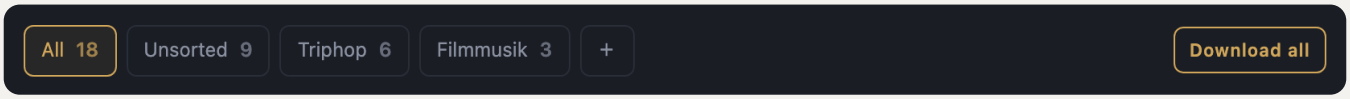
A song in the library



1. **Play** · play, with waveform at the bottom
2. **Star** · mark as favorite
3. **WAV** · download, 48 kHz, 16 bit
4. **TXT** · show the model's lyrics (only with vocals)
5. **Folder** · move to a folder
6. **Plus** · another version with a new seed
7. **Arrow** · copy the settings to the form
8. **Cross** · delete (press twice), cancel running

Click a title to rename it. The line below shows **BPM · IT** (iterations) **· VAR** (variance) · key · length. “Created in …” disappears once you play the song for the first time.

Folders



Create a folder with **+**. Drag songs onto a folder or move them with the folder button of the song row, each version separately. Clicking a folder shows only its songs; new songs land in the folder that is currently open. **Download all** or **Download folder** downloads the finished songs of the view as a ZIP. Deleting a folder (press twice) removes only the folder, not the songs.

Where the songs are

Finished songs are WAV files in the subfolder **data/songs** of the program folder, i.e. **~/MusicGenerator/data/songs** on the Mac or **C:\Users\<Name>\MusicGenerator\data\songs** on Windows. The file names are IDs, the titles live in the app. A different place: **Gear › Song folder**, enter a full path and save. The app creates the folder and moves the existing songs there; leaving it empty means back to default.

Optional: XL sound model

The installer asks once whether to also download the larger sound model **ACE-Step 1.5 XL** (4 instead of 2 billion parameters, 5.3 GB, Enter = no). According to the developers it sounds better, but takes about twice as long: on an M4 with 16 GB a 30-second song took 99 instead of 49 seconds. Recommended from 16 GB RAM or 12 GB graphics memory. Switch under **Gear › Sound model**, Standard stays preselected. To add it later, run Install again and choose XL, or use **--xl** (Mac) or **-XL** (Windows).

Updating

In the app: **Gear › Check for updates › Update now**. The app fetches the new version from GitHub and restarts, your songs are kept. Then OFF and ON once, so the model servers restart too. If it reports a new model version, run **Install** once.

Help

If something goes wrong

Problem	Solution
“Not enough disk space”	Free up space (empty the Trash!), then run Install again.
Download interrupted	Run Install again, the download resumes where it stopped.
“OFFLINE” at the top of the app	Music Generator OFF, then ON. If that doesn't help: details in the data folder (log files). Windows cases on page 5.
Song ends with an error	Hover over the error to see the reason. Retry with the circular arrow next to it.
Generating stalls	Sleep mode: the app keeps the computer awake; otherwise update.
Very slow	Lower the iterations (e.g. 30) or the length; old Windows cards are slower.
Noise instead of music	Mac with 8 GB, songs over 30 s: install the update, then OFF and ON.
Sound distorted	Gear › “Language model runs on” › CPU only, then OFF and ON.

Windows only

Problem	Solution
Model server doesn't start, message about a missing DLL	Update the NVIDIA driver (GeForce Experience or nvidia.com), then OFF and ON.
“ErrorOutOfDeviceMemory” in the log, song ends with an error	Not enough graphics memory: run Install again and choose a smaller language model, or halve the value VAE_CHUNK in data\engine.conf (e.g. 64). Then OFF and ON.
Installer stops at “Python packages”	Run **uv python install 3.12** in PowerShell and start Install again.
Norton etc. blocks ace-server.exe (“IDP.Generic”)	Usually a false alarm (file new, rare, unsigned). Restore it from quarantine, exclude the program folder from scanning, if in doubt check it on virustotal.com.

Model, license and training data

The music is generated by **ACE-Step 1.5**, an open model under the MIT license; according to the developers, the results may also be used commercially. The developers state it was trained on licensed, public-domain and synthetic music. The dataset itself is not published, so this claim cannot be verified. The developers ask you to check generated pieces for stylistic closeness and to disclose the use of AI.

Uninstalling

Double-click **Uninstall** (Mac-Uninstall.command, Windows-Uninstall.bat, Linux-Uninstall.sh). It asks whether to keep songs and library and removes models, environment and settings. Then delete the **MusicGenerator** folder. Helper tools such as uv stay installed.

EXPERIMENTAL, FEEDBACK WELCOME

Installation on Linux Mint and Ubuntu

1 Get the project and preview the steps

Open a terminal (**Ctrl+Alt+T**). If git is missing, run **sudo apt install git** first. Then get the project and use **--dry-run** to only show what the installer would do (nothing is changed):

```
git clone https://github.com/rabbitfiremediacreation/music-generator.git ~/MusicGenerator
cd ~/MusicGenerator
./ubuntu.sh --dry-run
```

The installer is **ubuntu.sh** in the project folder. For Arch Linux and derivatives use arch_linux.sh instead.

2 Install and start

The installer uses **apt** and asks for your password (sudo). It picks the backend itself: CUDA on NVIDIA if the CUDA toolkit is already installed, Vulkan on AMD, Intel or NVIDIA graphics with a Vulkan driver, otherwise CPU. Building the model server takes a few minutes, then the models are downloaded.

```
./ubuntu.sh --install-only
./ubuntu.sh --start
```

Then open **http://localhost:8765** in the browser. **--backend cuda|vulkan|cpu** forces a backend, **--lm 0.6B** picks a smaller language model, **./ubuntu.sh --stop** quits.

> **Linux:** So far tried by two users on **Linux Mint** (AMD graphics with Vulkan was fast, once with notes on playback quality). Untested on other hardware. Please report problems. A desktop app as AppImage is in progress: github.com/rabbitfiremediacreation/music-generator-desktop